Record Editing

Relevant source files

- h4styles.css
- hclient/core/hapi.js
- hclient/core/recordset.js
- hclient/core/utils.js
- hclient/core/utils_dbs.js
- hclient/core/utils_msg.js
- hclient/core/utils_ui.js
- hclient/framecontent/recordEdit.php
- hclient/widgets/editing/editing2.js
- hclient/widgets/editing/editing_exts.js
- hclient/widgets/editing/editing_input.js
- hclient/widgets/entity/manageDefDetailTypes.js
- hclient/widgets/entity/manageDefRecStructure.js
- hclient/widgets/entity/manageDefRecTypes.js
- hclient/widgets/entity/manageDefTerms.js
- hclient/widgets/entity/manageEntity.js
- hclient/widgets/entity/manageRecords.js
- hclient/widgets/search/search.js
- hclient/widgets/search/search_faceted.js
- hclient/widgets/search/search_faceted_wiz.html
- hclient/widgets/search/search_faceted_wiz.js
- hclient/widgets/search/svs_edit.html

`hclient/widgets/search/svs_edit.js`

`hclient/widgets/search/svs_list.js`

`hclient/widgets/viewers/resultList.js`

The Record Editing system in Heurist is responsible for creating, modifying, and validating records in the database. It provides a comprehensive user interface with specialized input controls for different field types, validation rules, and integration with other Heurist components.

Overview

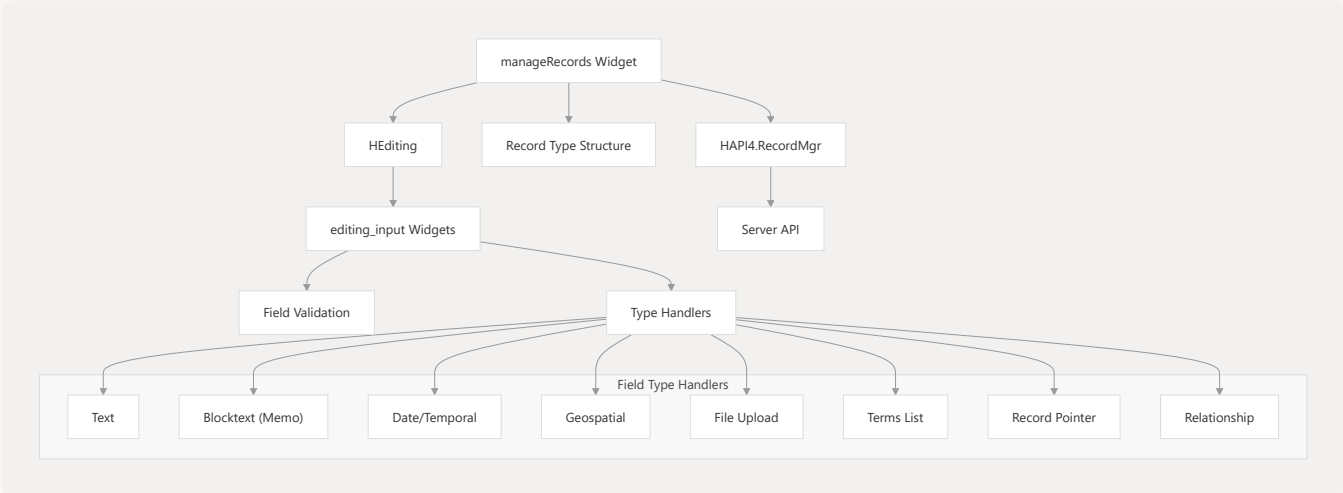
The Record Editing system allows users to interact with Heurist's database through intuitive forms that adapt based on record type structure. It supports a wide range of field types including text, numbers, dates, geographic coordinates, files, vocabulary terms, and relationships between records.

For information on how records are displayed to users (rather than edited), see Record Viewing.

Architecture

The Record Editing system consists of several key components that work together to provide a comprehensive editing experience.

Component Architecture



Sources: `hclient/widgets/entity/manageRecords.js` `1-470`

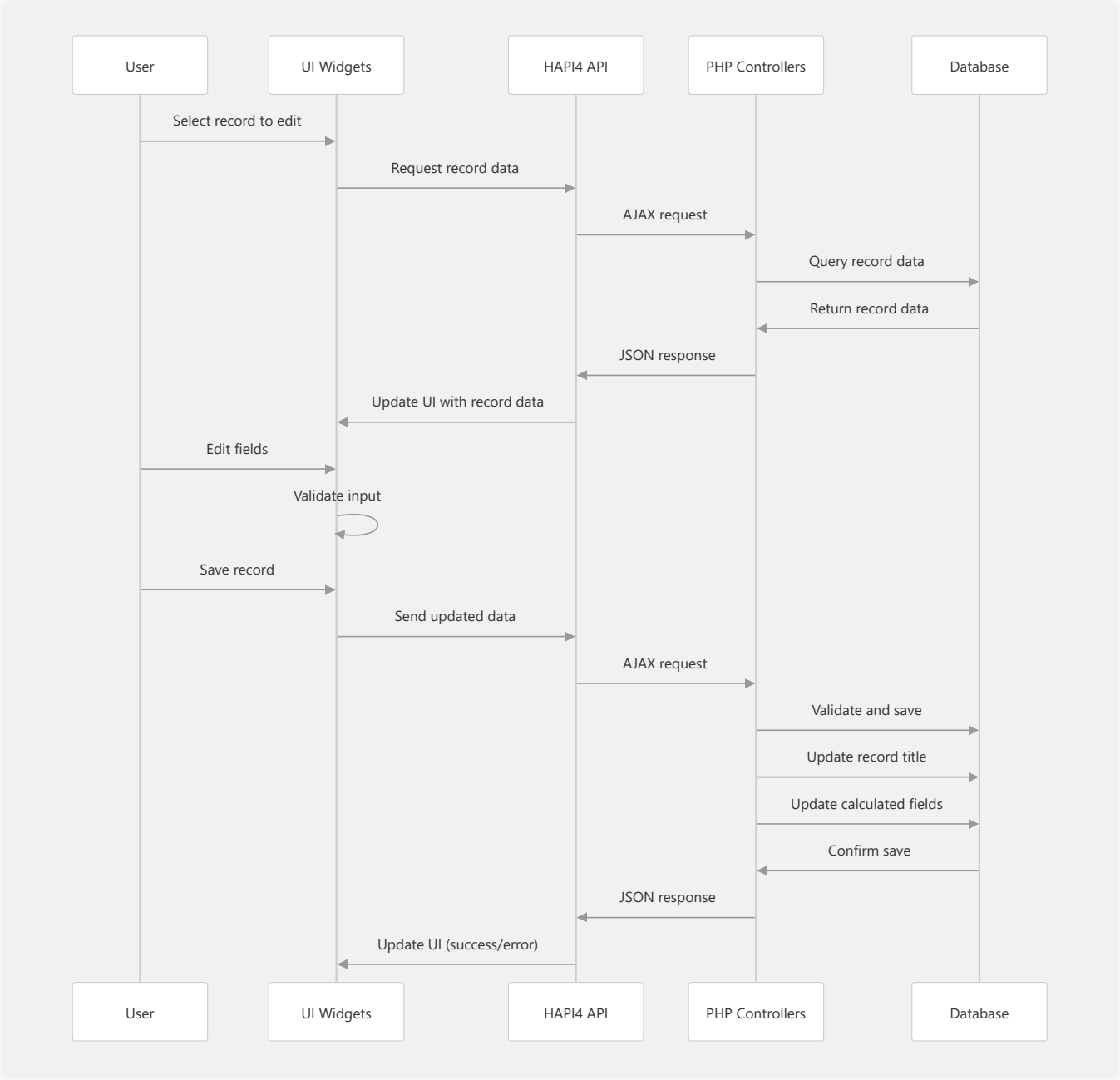
`hclient/widgets/editing/editing_input.js` `1-100`

Key Components

1. **manageRecords Widget**: The main controller for the record editing interface that manages the overall editing process.

2. **HEditing**: Manages the form structure, tracks modified fields, and coordinates validation.
3. **editing_input Widget**: Creates and manages individual input controls based on field type.
4. **Field Type Handlers**: Specialized code for different field types (text, date, geo, etc.).
5. **HAPI4.RecordMgr**: Handles server communication for loading and saving records.

Data Flow for Record Editing



Sources: `hclient/widgets/entity/manageRecords.js` `449-478` `hclient/core/hapi.js` `454-496`

editing_input Widget

The `editing_input` widget is the core component responsible for rendering and managing individual form fields. It creates appropriate input controls based on field type, handles validation, and manages repeatable fields.

Field Creation Process

The widget follows these steps to create input fields:

1. Determine field description and properties from record structure
2. Create appropriate input controls based on field type
3. Apply default values for new records
4. Set up validation rules and event handlers
5. Configure UI for repeatable fields if needed

```
// Creating an editing_input widget
$("<div>").editing_input({
    dtID: 1,                    // Detail type ID
    rectypeID: 10,              // Record type ID
    values: ["Initial value"], // Field values
    readonly: false,            // Whether field is readonly
    suppress_repeat: false,     // Allow repeating values
    change: onChangeCallback    // Change event handler
});
```

Sources: `hclient/widgets/editing/editing_input.js` `23-193`

Field Type Support

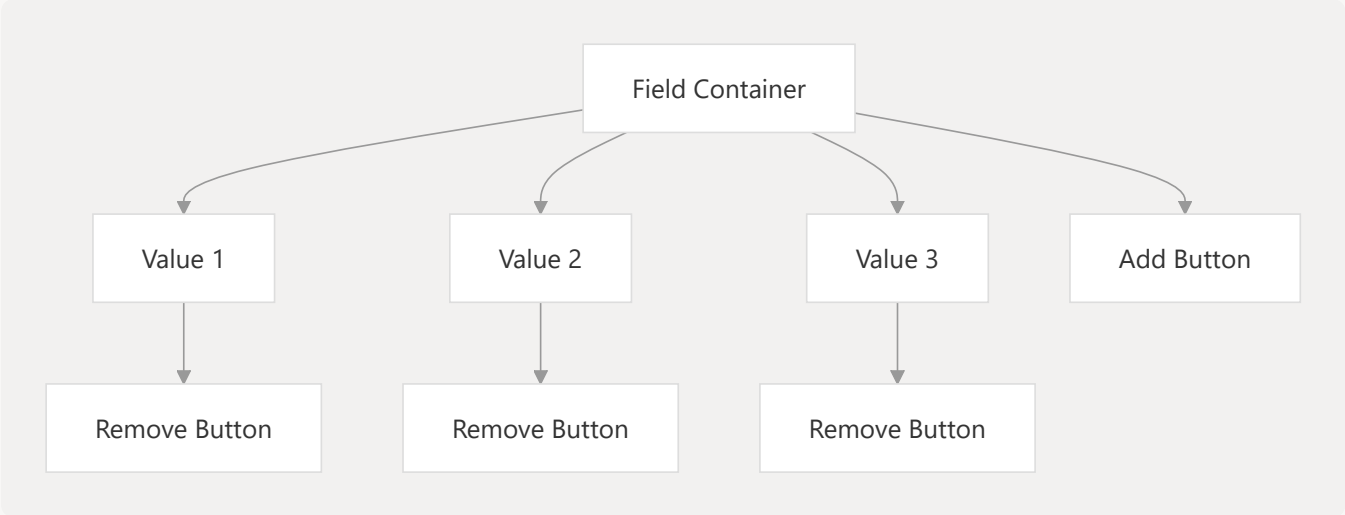
The widget supports various field types, each with specialized input controls:

Field Type	Widget	Description
freetext	Text input	Single-line text field
blocktext	Textarea/TinyMCE	Multi-line text with formatting options
float	Numeric input	Numbers with decimal places
date	Date picker	Date selection with calendar
geo	Map interface	Geographic coordinates entry
file	File uploader	File upload with preview
enum	Select/Radio/Checkbox	Selection from term vocabulary
resource	Record selector	Links to other records
relmarker	Relationship selector	Creates relationships between records
separator	Heading	Visual separator with no data

Sources: `hclient/widgets/editing/editing_input.js 246-323` `hclient/core/utils_dbs.js 81-97`

Repeatable Fields

Fields can be configured to allow multiple values. The `editing_input` widget provides UI controls for adding, removing, and reordering values.



Key features of repeatable fields:

1. "Add" button to create new input controls
2. "Remove" button for each value
3. Drag-and-drop reordering (optional)
4. Maximum value limit based on field configuration
5. Different display styles for different field types

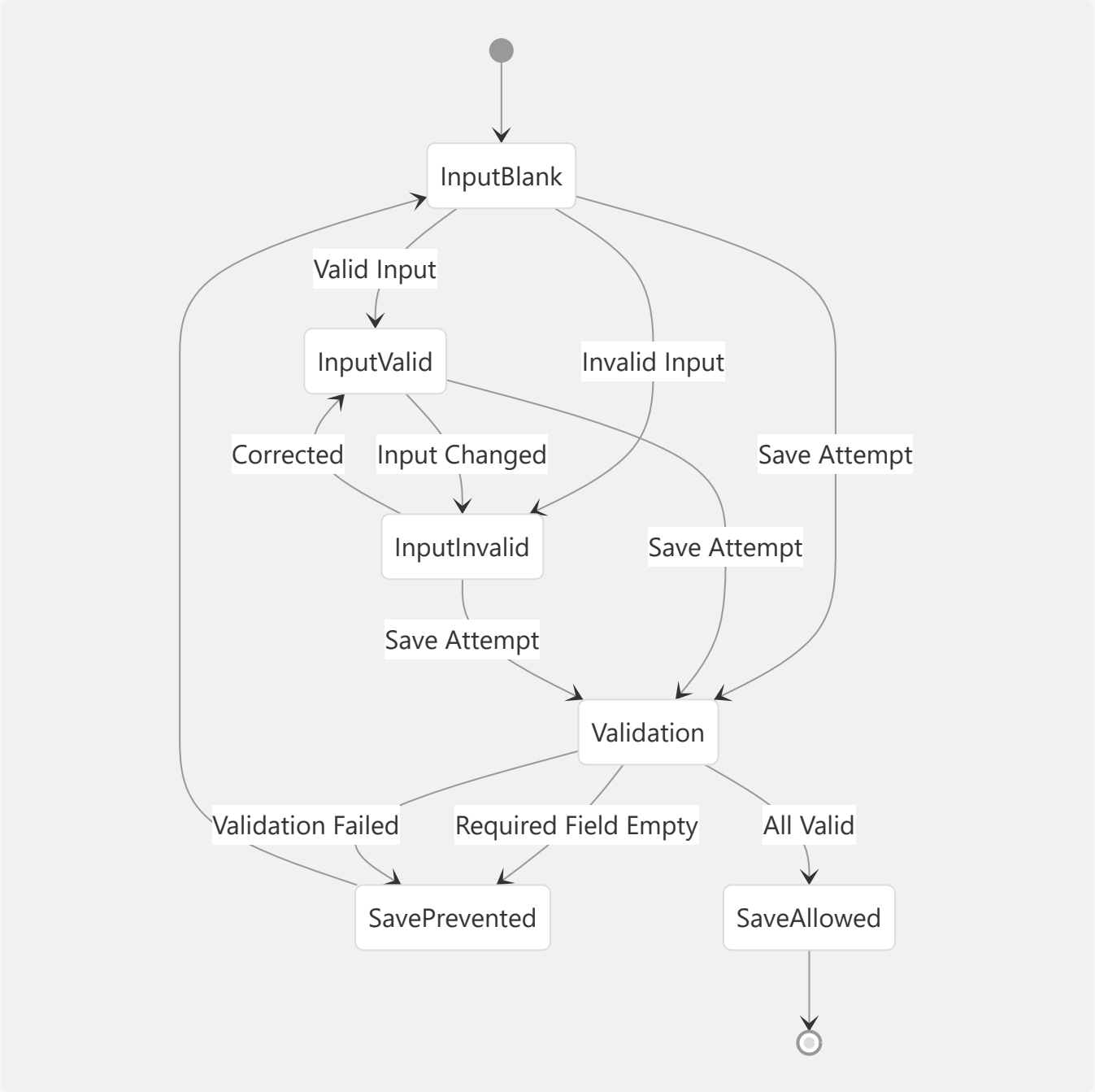
Sources: `hclient/widgets/editing/editing_input.js 243-310`
`hclient/widgets/editing/editing_input.js 808-470`

Field Requirements and Validation

Fields can have different requirement types that affect validation:

- **Required**: Must have a value to save the record
- **Recommended**: Suggested but not required
- **Optional**: Can be left empty
- **Hidden**: Not shown in the interface

The system provides visual cues for required fields and validation ensures that required fields have values before saving.



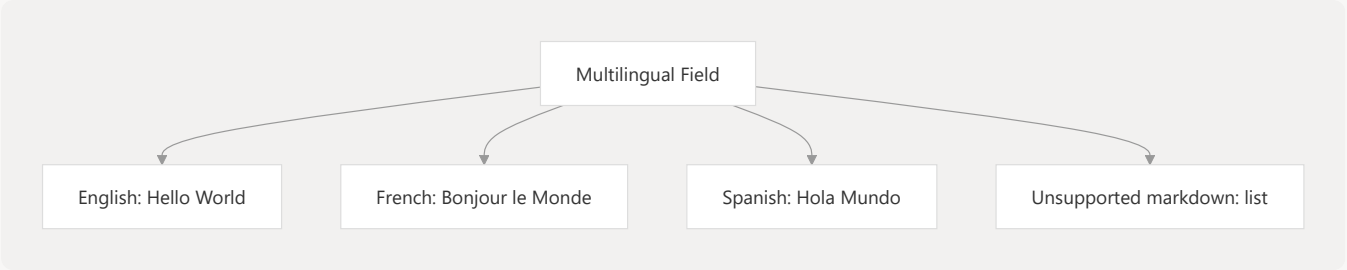
Sources: `hclient/widgets/editing/editing_input.js` `183-197`

`hclient/widgets/entity/manageRecords.js` `449-478`

Special Features

Multilingual Support

The system supports multilingual content through field translations. Users can add translations for field values in different languages.



Sources: `hclient/widgets/editing/editing_input.js` `301-396`

Structure Modification Interface

When editing records with admin privileges, users can modify the record type structure directly from the editing interface:

1. Gear icons appear beside fields
2. Clicking the gear opens a menu with options:
   - Edit field properties
   - Insert new field
   - Insert separator
   - Modify requirement type
   - Change field width

Sources: `hclient/widgets/entity/manageRecords.js` `183-241`
`hclient/widgets/entity/manageRecords.js` `482-568`

Workflow Stages

Fields can be designated as workflow stage fields, which control the processing status of records through predefined stages.

Sources: `hclient/widgets/editing/editing_input.js` `135-141`

Discouraged Editing

Fields can be marked as "discouraged" for editing, requiring confirmation before changes are allowed.

Sources: `hclient/widgets/editing/editing_input.js` `564-616`

Field Types and Input Controls

Text Fields

Two types of text fields are available:

- **freetext**: Single-line text input
- **blocktext**: Multi-line text with formatting options (uses TinyMCE editor)

Sources: `hclient/widgets/editing/editing_input.js` `294-295`

Date/Temporal Fields

Date fields provide a specialized interface for entering and validating dates, including:

- Calendar picker
- ISO date format validation
- Support for temporal objects with start/end dates

Sources: `hclient/widgets/editing/editing_input.js` `718-730`

Geographic Fields

Geo fields provide a specialized interface for entering geographic coordinates:

- Map-based point selection
- Direct coordinate entry
- Validation of coordinate formats

Sources: `hclient/widgets/editing/editing_input.js` `724`

File Upload Fields

File fields allow uploading and managing files:

- File selection dialog
- Upload progress indication
- Thumbnail preview for images
- File type validation

Sources: `hclient/widgets/editing/editing_input.js` `825-835`

Term Selection Fields

Term fields allow selecting from vocabularies:

- Dropdown selection
- Radio button or checkbox selection
- Hierarchical term display
- Term code display

Sources: `hclient/widgets/editing/editing_input.js 719` `hclient/core/utils_ui.js 297-359`

Record Pointer Fields

Record pointer fields create links to other records:

- Record search and selection
- Record creation from pointer field
- Display of linked record title

Sources: `hclient/widgets/editing/editing_input.js 416-424`

Relationship Marker Fields

Relationship fields create defined relationships between records:

- Selection of target record
- Selection of relationship type
- Directional relationship support

Sources: `hclient/widgets/editing/editing_input.js 422-428`

Integration with Other Components

The Record Editing system integrates with other Heurist components:

1. **Record Type Structure**: Defines fields and their properties
2. **Terms/Vocabularies**: Provides values for enumeration fields
3. **File Management**: Handles file uploads and storage
4. **Search System**: Enables finding records for relationship fields
5. **Mapping System**: Supports geospatial field input

Sources: `hclient/widgets/entity/manageRecords.js 1-30` `hclient/core/utils_ui.js 1-36`

Record Editing Process

The record editing process involves several steps:

1. **Initialization**:
   - `manageRecords` initializes with existing record ID or for new record
   - Record type structure is loaded to determine field layout
   - For existing records, data is fetched from the server
2. **Form Creation**:
   - `HEditing` creates the form structure
   - `editing_input` widgets are created for each field
   - Default values are applied for new records
3. **User Interaction**:
   - Users modify field values through the input widgets
   - Validation occurs on change or blur events
   - Repeatable fields can have values added/removed
4. **Validation**:
   - Required fields are checked
   - Field-specific validation rules are applied
   - Error messages are displayed to the user
5. **Saving**:
   - Modified field values are collected
   - Data is formatted for the server
   - Server validates and saves the data
   - Confirmation or error messages are returned

Sources: `hclient/widgets/entity/manageRecords.js` `449-478` `hclient/core/hapi.js` `454-496`

Using the Record Editor Programmatically

The Record Editor can be initialized programmatically:

```
// Open record editor for an existing record
window.hWin.HEURIST4.ui.openRecordEdit(recID, options);

// Open record editor for a new record
window.hWin.HEURIST4.ui.openRecordEdit(null, {
    rectypeID: 10,       // Record type ID
```

```
    samewindow: true,    // Open in same window
    onclose: callback    // Callback on close
});
```

The editor can be configured with various options including:

- Record type for new records
- Default values for fields
- Callback functions for events
- UI appearance settings

Sources: `hclient/core/utils_ui.js` `55-58`

Conclusion

The Record Editing system provides a flexible and powerful interface for creating and modifying records in Heurist. It supports a wide range of field types, complex validation rules, and integrates with other components of the system to provide a comprehensive editing experience.

For information about the database structure that underpins the record editing system, see Database Structure.